Universidad Icesi

# Cierre de Seguridad REST & JWT

Sesión 3: Ensamble Dual, Autorización RBAC con @PreAuthorize, Errores JSON y Forense Postman

**Computación en Internet II — Universidad Icesi**

Universidad ICESI

# Ruta de Aprendizaje — 4 Micro-Ciclos de Cierre (Teoría ↔ Práctica)

## Primera Mitad: Corrección Dual y Errores REST

### 01. Diagnóstico de Quiebre & Ajustes del Filtro

Por qué `@Order(1)` sin matcher secuestra rutas web y ajuste de `UserDetails` como principal.

### 02. Ensamble Dual & Manejo de Errores JSON

`securityMatcher("/api/**")`, `addFilterBefore` y respuestas HTTP nativas 401/403 en JSON.

**Enfoque Práctico:** Blindamos el pipeline REST desacoplándolo totalmente de las redirecciones de sesión HTML de Thymeleaf.

## Segunda Mitad: Autorización y Producción

### 03. Autorización RBAC con @PreAuthorize

Aseguramiento granular en `UsuarioRestController` con `hasRole()` y permisos atómicos `hasAuthority()`.

### 04. Laboratorio Postman & Sabotaje de Firma

Suite de 5 pruebas: anónimo (401), login y jwt.io, estudiante (403), admin CRUD y sabotaje HMAC.

### 05. Hidratación BD vs. 100% Stateless & Stale Claims

Dilema de roles obsoletos, revocación en vivo, consulta BD en filtro y Refresh Tokens.

Universidad ICESI

# 01. Diagnóstico de Quiebre y Ajustes del Filtro

¿Por qué la cadena @Order(1) sin securityMatcher secuestra el tráfico web de la aplicación?

Universidad ICESI

# El Secuestro de Rutas: ¿Por qué colapsa el MVC sin securityMatcher?

## Anatomía del Error: Selección de Cadena en FilterChainProxy

La ausencia de un RequestMatcher restrictivo en la primera cadena causa el bloqueo total de rutas HTML

ESTADO INICIAL (CON ERROR)

### @Order(1) sin .securityMatcher()

Al no tener matcher, asume `/**` por omisión.

```
// Petición desde Navegador:
GET /login o GET /usuarios
➔ Atrapado por Cadena 1 (STATELESS)
```

**CONSECUENCIA INMEDIATA:**

- No hay sesión `JSESSIONID` permitida.
- No hay formulario `formLogin()` configurado.
- **La cadena @Order(2) MVC queda muerta.**
- El usuario recibe 403 Forbidden o error blanco.

SOLUCIÓN ARQUITECTÓNICA

### .securityMatcher("/api/**") Explícito

Delimita con precisión quirúrgica el dominio REST.

```
// Bifurcación Inteligente:
GET /api/** ➔ Cadena 1 (@Order 1)
GET /** (MVC) ➔ Cadena 2 (@Order 2)
```

**BENEFICIO EN PRODUCCIÓN:**

- La API REST opera 100% Stateless con JWT.
- Thymeleaf mantiene sus cookies y formulario.
- **Cero colisiones entre ambos paradigmas.**
- Coexistencia limpia en un único monolito Spring.

# Ajuste en JwtAuthenticationFilter: Hidratando el Principal UserDetails

### 1. src/.../security/jwt/JwtAuthenticationFilter.java

```java
// En doFilterInternal():
final String userEmail = jwtService.extractUsername(token);

if (userEmail != null &&
    SecurityContextHolder.getContext().getAuthentication() == null) {

    UserDetails userDetails =
        userDetailsService.loadUserByUsername(userEmail);

    if (jwtService.isTokenValid(token, userDetails)) {
        // AJUSTE CRÍTICO:
        // Pasar el objeto userDetails completo como principal,
        // NO únicamente el String del email
        UsernamePasswordAuthenticationToken authToken =
            new UsernamePasswordAuthenticationToken(
                userDetails,            // <-- Principal Tipado
                null,                   // Credentials (null en JWT)
                userDetails.getAuthorities() // Authorities
            );

        authToken.setDetails(
            new WebAuthenticationDetailsSource().buildDetails(request));
        SecurityContextHolder.getContext().setAuthentication(authToken);
    }
}
```

### 2. Justificación Arquitectónica del Cambio

**Soporte para @AuthenticationPrincipal**

Permite inyectar directamente `@AuthenticationPrincipal UserDetails user` en los métodos de cualquier controlador REST.

**Compatibilidad con SpEL en @PreAuthorize**

Expresiones como `authentication.principal.username` acceden a métodos reales del objeto de seguridad sin casteos riesgosos.

**Consistencia de Contexto**

El hilo de Tomcat preserva exactamente la misma estructura de identidad tanto en la cadena REST como en la cadena MVC.

Universidad ICESI

# 02. Ensamble Definitivo de la Dual FilterChain y Errores JSON

Configuración de SecurityConfig, eliminación de redirecciones HTML 302 y manejo de 401/403 en JSON.

Universidad ICESI

# Pipeline de Excepciones REST: Eliminando Redirecciones HTML (302)

## Despacho de Errores: Devolviendo JSON Puro a Clientes Desacoplados

Configuración de AuthenticationEntryPoint y AccessDeniedHandler en la cadena REST



Universidad ICESI

# Implementación Definitiva: apiSecurityFilterChain en SecurityConfig

## 1. Declaración de Cadena 1 (@Order(1))

```
@Bean
@Order(1)
public SecurityFilterChain apiSecurityFilterChain(HttpSecurity http)
        throws Exception {
    return http
        // 1. Aislar únicamente endpoints bajo /api/**
        .securityMatcher("/api/**")
        // 2. Desactivar CSRF (Stateless / sin cookies)
        .csrf(csrf -> csrf.disable())
        // 3. Sesiones 100% en token (cero RAM en Tomcat)
        .sessionManagement(s -> s
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        )
        // 4. Reglas públicas vs protegidas
        .authorizeHttpRequests(authz -> authz
            .requestMatchers("/api/v1/auth/**").permitAll()
            .anyRequest().authenticated()
        )
        // ... continua en columna derecha ...
```

## 2. Excepciones JSON y Registro del Filtro

```
        // 5. Manejadores de error nativos en JSON
        .exceptionHandling(ex -> ex
            .authenticationEntryPoint((req, res, err) -> {
                res.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
                res.setContentType("application/json");
                res.getWriter().write(
                  "{"error": "Token JWT ausente o invalido"}");
            })
            .accessDeniedHandler((req, res, err) -> {
                res.setStatus(HttpServletResponse.SC_FORBIDDEN);
                res.setContentType("application/json");
                res.getWriter().write(
                  "{"error": "Permisos insuficientes"}");
            })
        )
        // 6. Posicionar filtro JWT antes de UsernamePassword
        .addFilterBefore(jwtAuthenticationFilter,
            UsernamePasswordAuthenticationFilter.class)
        .build();
}
```

# Comparativa de Contratos: Cadena 1 (REST) vs. Cadena 2 (MVC)

### Cadena 1: @Order(1) — REST API (/api/**)

- **Matcher:** `/api/**` (Aisla la API móvil y SPA).
- **Gestión de Sesión:** `STATELESS` (Cero memoria en servidor).
- **Mecanismo de Auth:** Header `Authorization: Bearer <token>`.
- **Protección CSRF:** Deshabilitada (Seguro por ser Stateless sin cookies).
- **Respuestas de Error:** Códigos HTTP `401` y `403` en payload JSON.
- **Filtro Personalizado:** `JwtAuthenticationFilter` registrado.

### Cadena 2: @Order(2) — Web MVC (/**)

- **Matcher:** `/**` por omisión (Captura vistas Thymeleaf).
- **Gestión de Sesión:** Stateful indexado por cookie `JSESSIONID`.
- **Mecanismo de Auth:** Formulario HTML clásico con `formLogin()`.
- **Protección CSRF:** Activa obligatoria para evitar ataques en formularios.
- **Respuestas de Error:** Redirección `302 Found` a `/login` o `/access-denied`.
- **Filtro Personalizado:** Ninguno; opera con la cadena base de Spring.

Universidad ICESI

# 03. Autorización Granular con @PreAuthorize y RBAC

De la identidad a los privilegios: consumo de Authorities e inyección declarativa con SpEL.

# Flujo de Autorización RBAC: De los Claims en el JWT a @PreAuthorize

## Secuencia de Autorización Sin Consulta a Base de Datos (Zero-DB Hit)

Cómo el claim "authorities" en el JWT alimenta la evaluación SpEL en cada método del Controller

**PASO 1: TOKEN ENTRANTE**

### Payload Claims

El cliente envía el JWT con los claims de roles

```
{
"sub": "admin@icesi...",
"authorities": [
 "ROLE_ADMIN",
 "USER_READ"
]
}
```

Firmado con HMAC-SHA256.
No requiere ir a la BD
para conocer los roles.

**PASO 2: FILTRO JWT**

### Hidratación Contexto

Extrae las autoridades y crea el token de Spri

**SimpleGrantedAuthority**

→ ROLE_ADMIN

→ USER_READ

Inyectado en:
**SecurityContextHolder**

Disponible para todo el
hilo HTTP actual en
forma inmutable.

**PASO 3: INTERCEPTOR AOP**

### @EnableMethodSecurity

Intercepta antes de invocar el método de Co

**Expresión SpEL:**

hasRole('ADMIN')

hasAuthority('READ')

**Evaluación:**
**TRUE → Acceso Concedido**

Si evalúa FALSE, arroja
AccessDeniedException
sin ejecutar el método.

**PASO 4: RESULTADO**

### Ejecución

Respuesta limpia:

**200 OK**
JSON con Datos

**403 Forbidden**
AccessDenied JSON

Máximo rendimiento y
desacoplamiento total.

# Habilitación Declarativa: Expresiones SpEL en @PreAuthorize

## 1. Roles (RBAC) vs. Permisos Atómicos

**hasRole('ADMINISTRADOR')**
Busca automáticamente la autoridad con prefijo `ROLE_ADMINISTRADOR`. Ideal para jerarquías amplias.

**hasAuthority('USER_READ')**
Evalúa el permiso directo sin asumir ningún prefijo. Permite un control de privilegios granular por acción.

## 2. Catálogo de Expresiones SpEL

| Expresión SpEL | Significado y Uso |
|---|---|
| `hasRole('ADMIN')` | Requiere `ROLE_ADMIN`. |
| `hasAnyRole('A', 'B')` | Permite acceso si posee el rol A o el rol B. |
| `hasAuthority('READ')` | Exige el permiso puntual `READ`. |
| `hasRole('A') or hasAuthority('B')` | Combina roles con permisos usando operadores booleanos. |
| `isAuthenticated()` | Cualquier usuario autenticado legítimamente. |

Universidad ICESI

# Asegurando UsuarioRestController con @PreAuthorize

## 1. Endpoints de Consulta (Lectura Flexible)

```
@RestController
@RequestMapping("/api/v1/usuarios")
@RequiredArgsConstructor
public class UsuarioRestController {

    // Lectura permitida a Admin, Profesor o quien tenga USER_READ
    @GetMapping
    @PreAuthorize("hasAnyRole('ADMINISTRADOR', 'PROFESOR') " +
      "or hasAuthority('USER_READ')")
    public List<usuarioresponsedto> obtenerUsuarios(
    @RequestParam(required = false) Boolean active) {
        return usuarioMapper.toListUsuarioResponseDTO(
            usuarioService.listarUsuarios(active));
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasAnyRole('ADMINISTRADOR', 'PROFESOR') " +
      "or hasAuthority('USER_READ')")
    public ResponseEntity<usuarioresponsedto> obtnerUsuarioPorId(
    @PathVariable Long id) {
        return usuarioService.obtenerPorId(id)
            .map(u -> ResponseEntity.ok(usuarioMapper.toUsuarioResponseDTO(u)))
            .orElse(ResponseEntity.notFound().build());
    }</usuarioresponsedto></usuarioresponsedto>
```

## 2. Endpoints de Mutación (Exclusivos Admin)

```
    // Creación, actualización y eliminación restringidas a ADMIN
    @PostMapping
    @PreAuthorize("hasRole('ADMINISTRADOR')")
    public ResponseEntity<usuarioresponsedto> crearUsuario(
    @RequestBody UsuarioRequestDTO dto) {
        // ... creación con 201 Created ...
    }

    @PutMapping("/{id}")
    @PreAuthorize("hasRole('ADMINISTRADOR')")
    public ResponseEntity<usuarioresponsedto> actualizarUsuario(
    @PathVariable Long id, @RequestBody UsuarioRequestDTO dto) {
        // ... actualización ...
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMINISTRADOR')")
    public ResponseEntity<void> eliminarUsuario(@PathVariable Long id) {
        usuarioService.eliminarUsuario(id);
        return ResponseEntity.noContent().build();
    }
}</void></usuarioresponsedto></usuarioresponsedto>
```

# 04. Laboratorio Forense en Postman y Sabotaje de Firma

Protocolo de 5 pruebas: verificación de permisos, inspección de claims y ruptura matemática del HMAC.

# Matriz de Pruebas en Postman (Casos 1, 2 y 3)

### 1. Petición sin Token ➔ 401 Unauthorized (JSON)

`GET /api/v1/usuarios` sin header `Authorization` responde 401 en JSON (ya no redirige al HTML `/login`).

### 2. Login con Rol Estudiante + Inspección en jwt.io

`POST /api/v1/auth/login` con credenciales de estudiante y decodificación de los *claims* en `jwt.io`.

### 3. Estudiante Invoca DELETE /usuarios/1 ➔ 403 Forbidden

El token es auténtico, pero carece del rol `ROLE_ADMINISTRADOR` exigido por `@PreAuthorize`.

```
// POST /api/v1/auth/login
{
  "correoInstitucional": "estudiante@icesi.edu.co",
  "password": "user123"
}

// Respuesta 200 OK:
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "type": "Bearer",
  "expiresIn": 3600000,
  "correoInstitucional": "estudiante@icesi.edu.co"
}
```

# Matriz de Pruebas en Postman (Casos 4 y 5: Admin & Sabotaje)

### 4. Login Admin y Operación CRUD Completa (200 / 201)

Autenticar como `admin@icesi.edu.co` con `admin123`. Ejecutar `GET /usuarios` y crear un usuario con `POST`. Acceso total concedido.

### 5. Sabotaje Criptográfico de Firma (401 Unauthorized)

Tomar el token de admin y alterar 4 caracteres de la firma en Postman. JJWT detecta la alteración, lanza `SignatureException` y rechaza la petición.

```
// Token Original de Administrador:
eyJhbGciOiJIUzI1NiJ9...W5v8

// Token Saboteado en Postman:
eyJhbGciOiJIUzI1NiJ9...xxxx

// Respuesta 401 del Servidor:
HTTP/1.1 401 Unauthorized
{"error": "No autenticado: Token JWT ausente, invalido o expirado"}
```

Universidad ICESI

# Anatomía Forense: ¿Por qué falla la firma alterada en JJWT?

## Verificación Criptográfica: Integridad Matemática Imposible de Falsificar

El servidor recalcula el HMAC del payload entrante contra la clave secreta privada jwt.secret

**PETICIÓN INTERCEPTADA EN TRÁNSITO**

**HEADER (H)**
`{"alg":"HS256"}`

•

**PAYLOAD MANIPULADO (P')**
`sub="estudiante" ➔ cambiado a "admin@icesi.edu.co"`

•

**FIRMA RECIBIDA (S)**
Firma original previa al cambio

### 1. RECÁLCULO EN SERVIDOR (JJTW 0.12+)

JJWT toma el Header y Payload recibido y calcula:

```
HMAC-SHA256(H + "." + P', jwt.secret)
➔ Hash_Calculado = "9e8a7b6c5d..."
```

La clave privada `jwt.secret` solo reside en la memoria del servidor. El atacante jamás podrá adivinar la firma correspondiente al nuevo payload alterado.

### 2. COMPARACIÓN CRIPTOGRÁFICA

Compara el hash calculado contra la firma enviada:

```
Hash_Calculado != Firma_Recibida (S)
➔ LANZA: io.jsonwebtoken.security.SignatureException
```

- **Catch en Filtro:** `SecurityContextHolder.clearContext()`
- **Respuesta:** `401 Unauthorized` inmediata.
- **Base de Datos:** Cero consultas ejecutadas (Zero-DB).

Universidad ICESI

# 05. Escenarios Críticos en Producción: Hidratación vs. Stateless

Vulnerabilidades reales, el dilema de los Stale Claims, Hidratación desde BD y persistencia stateless.

# Tres Escenarios Críticos de Seguridad JWT en Producción

## 1. ROBO DE TOKEN VIGENTE

**El Riesgo Stateless**
Si un atacante roba un JWT válido (por XSS), el servidor lo aceptará hasta que llegue su fecha `exp`.

**Mitigación:** Emitir *Access Tokens* de vida corta (5–15 min) + *Refresh Tokens* en cookie `HttpOnly` o lista de revocación en Redis.

## 2. ALTERACIÓN DEL PAYLOAD

**Escalación de Privilegios**
Un estudiante decodifica su JWT en Base64, cambia `ROLE_ESTUDIANTE` por `ROLE_ADMINISTRADOR` y lo reenvía.

**Defensa:** Al no conocer la `jwt.secret`, el HMAC calculado falla (`SignatureException` ➔ `401 Unauthorized`).

## 3. STALE CLAIMS (DATOS VIEJOS)

**Revocación de Rol en Vivo**
¿Qué pasa si desactivamos un usuario cuyo token aún tiene 45 min de vida?

**Defensa por Hidratación BD:** Al consultar `loadUserByUsername(email)` en el filtro, obtenemos permisos frescos de la BD al instante.

# Decisión de Arquitectura: Hidratación desde BD vs. 100% Claims del Token

| Dimensión Arquitectónica | Enfoque A: Hidratación de UserDetails desde BD | Enfoque B: 100% Stateless Leyendo Claims del JWT |
|---|---|---|
| **Costo en Base de Datos** | 1 consulta rápida por clave única ( `username` / `email` ) | **0 consultas SQL** (Solo verificación criptográfica en CPU) |
| **Revocación de Roles / Permisos** | **Efecto Inmediato** en la siguiente petición HTTP | Diferido hasta que el token expire ( `exp` ) |
| **Desactivación de Cuenta (active=false)** | **Bloqueo Inmediato** al evaluar el estado del usuario | El usuario sigue operando hasta que expire su JWT |
| **Caso de Uso Recomendado** | Sistemas académicos, financieros y empresariales críticos | Microservicios masivos con Access Tokens de 5 minutos |

**Balance de Ingeniería:** Incluso consultando `UserDetailsService` , la arquitectura sigue siendo **100% Stateless en la capa web** porque el contenedor no almacena ningún objeto `HttpSession` en RAM entre peticiones.

# Estrategias Complementarias: Refresh Tokens y Blacklist Redis

### El Problema (Zero-DB)

Al confiar 100% en la firma sin ir a la base de datos, el servidor asume que los roles del token siguen vigentes.

Si un administrador suspende a un empleado a las **10:05 AM** pero su token vence a las **11:00 AM**, podrá seguir ejecutando peticiones durante **55 minutos**.

### Solución 1: Refresh Tokens

- **Access Token (JWT):** Expiración muy corta (ej. 5 a 15 min). Viaja en cada llamada HTTP.

- **Refresh Token:** Cadena opaca guardada en base de datos (duración 30 días). Se usa solo para emitir nuevos JWTs y puede revocarse inmediatamente al despedir al usuario.

### Solución 2: Redis Blacklist

Al cerrar sesión o revocar permisos, el identificador único del token (`jti`) se guarda en una BD en memoria ultrarrápida (Redis).

La clave en Redis tiene un tiempo de vida (TTL) igual a los minutos restantes del token, eliminándose sola sin saturar RAM.

# Verificación de Convivencia MVC y Checklist Final

## Prueba de Fuego en Navegador Web

### 1. Abrir Navegador en /usuarios

Ingresar a `http://localhost:8080/springboot-api/usuarios`. El navegador redirige limpiamente a `/login` (Thymeleaf).

### 2. Iniciar Sesión con Formulario

Ingresar con `admin@icesi.edu.co` / `admin123`. Comprobar que se crea la cookie `JSESSIONID` en Storage.

**Conclusión:** El sistema atiende peticiones REST 100% Stateless y vistas HTML Stateful de forma simultánea y transparente.

## Checklist Arquitectónico Completado

- **Dual FilterChain Operativa**
  `@Order(1)` con `securityMatcher("/api/**")` y `@Order(2)` para MVC.
- **Seguridad Declarativa RBAC**
  Endpoints protegidos con `@PreAuthorize` mediante roles y authorities.
- **Respuestas Nativas en JSON**
  401 y 403 formateados en JSON sin fugas de redirecciones web.
- **Integridad Criptográfica Verificada**
  Pruebas en Postman corroborando el rechazo ante sabotajes de firma.

Universidad ICESI

# Fin de la Sesión 3 — Cierre de REST & JWT

¡Arquitectura Híbrida 100% Funcional: REST Stateless con JWT y Web MVC en Convivencia Total!